



Universidad Nacional Experimental de Guayana Vicerrectorado Académico

Proyecto de carrera: Ingeniería en informática

Asignatura: Lenguajes y Compiladores.

Sección 1.

Lenguajes y Gramáticas Formales

Profesor:

Felix Marquez

Alumnos:

Mauricio Leal 30.366.382

Santiago Sanchez 30.001.012

Nicole García 30.809.865

Jesús Ramírez 31.074.731

Ciudad Guayana, Mayo de 2026.

RESUMEN

El análisis presentado en este documento aborda las gramáticas y lenguajes formales, esenciales en la teoría de la computación, y su clasificación en la jerarquía de Chomsky. Se estudian técnicas como la eliminación de ambigüedades y la recursividad por la izquierda, necesarias para optimizar el procesamiento sintáctico. A través de ejemplos prácticos, se demuestra su aplicación en la construcción de autómatas finitos, la validación de datos con expresiones regulares, y su uso en lenguajes de programación y estrategias de ajedrez. Se destaca su impacto en áreas como el análisis de datos, evidenciando su relevancia en el desarrollo de soluciones computacionales avanzadas. Además, el estudio de los lenguajes formales es clave en la implementación de compiladores, donde se utilizan técnicas de análisis léxico y sintáctico para convertir el código fuente en instrucciones ejecutables por una máquina. La capacidad de representar estructuras sintácticas de manera formal ha permitido avances en inteligencia artificial y procesamiento del lenguaje natural, facilitando la interacción entre humanos y sistemas computacionales.

INTRODUCCION

En el ámbito de la ciencia de la computación, la construcción de software complejo y la definición de lenguajes de programación requieren de un marco teórico robusto que garantice la precisión, la predictibilidad y la eficiencia en el procesamiento de la información. La Teoría de Autómatas y los Lenguajes Formales constituyen la piedra angular de esta disciplina, proveyendo los modelos matemáticos necesarios para entender cómo las máquinas procesan la sintaxis y la semántica de las instrucciones.

Es importante abordar los componentes esenciales de este ecosistema teórico-práctico a través de cuatro ejes fundamentales. En primer lugar, se analiza la Relación Gramática-Lenguaje, detallando el mecanismo de derivación mediante el cual una estructura abstracta es capaz de generar un lenguaje formal computable. Posteriormente, se examina la Jerarquía de Chomsky, clasificando las gramáticas desde los modelos más restrictivos hasta los universales, utilizando la notación Backus-Naur Form (BNF). En el aspecto práctico, se explora el Modelado de Gramáticas Libres de Contexto (GLC) aplicadas a un sistema gráfico vectorial simulado mediante caracteres alfabéticos. Asimismo, se profundiza en la Higiene y Optimización de Gramáticas, un bloque crítico para el diseño de compiladores donde se identifican y corrigen patologías sintácticas comunes como la ambigüedad, la recursividad por la izquierda y la necesidad de factorización. A través de este recorrido, se busca demostrar cómo los conceptos puramente matemáticos se transforman en herramientas de ingeniería de software indispensables para el desarrollo de analizadores léxicos y sintácticos en los compiladores modernos.

2.1.- Fundamentos y Jerarquía (Teoría Aplicada)

2.1.1.- Relación Gramática-Lenguaje

Un lenguaje formal es un lenguaje artificial que, a diferencia de los lenguajes naturales, no está centrado en la comunicación humana habitual, sino en la definición y aplicación de una serie de reglas específicas en un entorno específico. Consta de una serie de símbolos básicos, que se combinan en cadenas para formar expresiones complejas.

Los símbolos constituyen el alfabeto de un lenguaje formal, mientras que las cadenas son las palabras. Tanto la combinación de los símbolos como la interpretación de las cadenas se realiza de acuerdo con reglas rigurosas, establecidas de antemano, que no admiten excepciones; en conjunto, estas reglas conforman la gramática del lenguaje, Noam Chomsky en una conferencia en Chicago en 1970 menciona que “El lenguaje es un proceso de creación libre; sus leyes y principios son fijos, pero la manera en que se utilizan los principios de generación es libre e infinitamente variada”.

Frente a la creatividad y complejidad del lenguaje natural, que permiten su uso en las más diversas situaciones, los lenguajes formales han sido diseñados para ser utilizados en contextos muy precisos. Su campo de aplicación incluye la lógica, las matemáticas, la informática y la lingüística.

No se deben confundir los lenguajes formales aquí tratados con el registro lingüístico formal (también llamado a veces lenguaje formal), correspondiente al uso cuidadoso y correcto del lenguaje, propio de ciertas situaciones comunicativas, opuesto al registro informal, en el que se emplean frases y expresiones coloquiales.

Relación

La gramática y el lenguaje formal están totalmente relacionados, ya que uno depende del otro, la gramática es el conjunto de reglas que define cómo deben organizarse los símbolos del alfabeto para formar cadenas válidas, es decir, palabras o expresiones dentro del lenguaje formal, entonces de tal manera en que sin la gramática, no se podría construir ni interpretar correctamente un lenguaje formal, podría decirse que la gramática

es la base de reglas del cómo se forma el lenguaje formal con instrucciones, etc. Y el lenguaje formal es el resultado de que esas reglas o instrucciones se cumplan en su totalidad.

Ejemplo

Consideremos el lenguaje de todas las cadenas que comienzan y terminan con la letra "a", y que solo contienen las letras "a" y "b".

El conjunto de cadenas de este lenguaje podría ser:

`{"a", "aa", "aba", "aabaa", "abbbba", "aabba", ...}`

2.1.2.- Jerarquía de Chomsky: Explique los 4 tipos de gramáticas de la Jerarquía de Chomsky. Para cada tipo, proporcione un ejemplo práctico representado en notación BNF (Backus-Naur Form).

La Jerarquía de Chomsky, propuesta por el lingüista Noam Chomsky, es un sistema que clasifica los lenguajes formales en cuatro tipos, basándose en la complejidad de las reglas gramaticales necesarias para generarlos. Esta jerarquía es fundamental en la teoría de la computación y en el diseño de compiladores, ya que nos permite entender qué tipo de "máquina" o autómatas se necesita para reconocer y procesar un lenguaje.

Los cuatro niveles de la jerarquía, del más restrictivo al más general son:

Tipo 3: Gramáticas Regulares

Son las gramáticas más simples. Generan los lenguajes regulares. Sus reglas de producción son muy estrictas: un no-terminal solo puede producir un terminal, o un terminal seguido de un único no-terminal.

Autómata: Autómata Finito (AF), ya sea determinista o no determinista.

Uso práctico: Se usan para definir la estructura de componentes léxicos como identificadores, números o palabras clave en un lenguaje de programación. Son la base de las expresiones regulares.

Tipo 2: Gramáticas Libres de Contexto (GLC)

Son más expresivas que las regulares. La parte izquierda de una producción siempre es un único símbolo no-terminal. Esto significa que la regla se puede aplicar "libremente", sin importar el contexto en el que aparezca el no-terminal.

Autómata: Autómata de Pila (Pushdown Automaton).

Uso práctico: Son la base para definir la sintaxis de la mayoría de los lenguajes de programación, como la estructura de las sentencias, bloques de código, expresiones aritméticas, etc.

Ejemplo en BNF: La clásica gramática para expresiones aritméticas balanceadas.

$\langle \text{expr} \rangle ::= \langle \text{expr} \rangle + \langle \text{termino} \rangle \mid \langle \text{termino} \rangle$

$\langle \text{termino} \rangle ::= \langle \text{termino} \rangle * \langle \text{factor} \rangle \mid \langle \text{factor} \rangle$

$\langle \text{factor} \rangle ::= (\langle \text{expr} \rangle) \mid \text{id}$

Tipo 1: Gramáticas Sensibles al Contexto

Las reglas de producción pueden tener más de un símbolo a la izquierda, pero con una restricción: la longitud del lado derecho debe ser al menos igual a la del lado izquierdo. Esto permite que una regla se aplique solo en un "contexto" específico.

Autómata: Autómata Linealmente Acotado (Linearly Bounded Automaton), que es una Máquina de Turing con una cinta de memoria limitada.

Uso práctico: Son menos comunes en la práctica de compiladores debido a su complejidad, pero pueden modelar relaciones como la declaración de una variable antes de su uso.

Ejemplo en BNF : Genera el lenguaje $\{a^n b^n c^n \mid n \geq 1\}$, que no es libre de contexto.

$S \rightarrow aSBC \mid aBC$

$CB \rightarrow BC$

$aB \rightarrow ab$

$bB \rightarrow bb$

$bC \rightarrow bc$

$cC \rightarrow cc$

Tipo 0: Gramáticas sin Restricciones (Recursivamente Enumerables)

Son las más generales. No tienen ninguna restricción en sus reglas de producción (salvo que la parte izquierda no puede ser vacía).

Autómata: Máquina de Turing.

Uso práctico: Tienen un gran poder expresivo, pero son tan complejas que su análisis es, en general, indecidible. Son principalmente de interés teórico para definir los límites de lo que es computable.

Ejemplo en BNF : No se suelen usar en la práctica, pero cualquier gramática de los tipos anteriores es también de Tipo 0. Un ejemplo teórico podría ser una gramática que simule los cálculos de una Máquina de Turing

Sección 2.2.- Derivación y Modelado (Caso Práctico: Genoma)

2.2.1.- Gramática para Dibujar Basada en una GLC

El modelado geométrico computacional mediante estructuras lingüísticas formales permite abstraer comportamientos físicos e interpretarlos sintácticamente. Para asociar el alfabeto molecular del genoma con instrucciones de dibujo, se implementa una correspondencia formal con el paradigma de L-Systems (Sistemas de Lindenmayer) y

Gráficas de Tortuga. El estado de la entidad dibujante (o "tortuga") queda determinado por un vector tridimensional (x, y, θ) , donde (x, y) representan las coordenadas cartesianas sobre el plano y θ denota el ángulo de orientación respecto al eje horizontal.

Se define formalmente la semántica de los elementos terminales del alfabeto Σ de la siguiente manera:

a : Avanzar una distancia fija d trazando una línea recta continua desde la posición actual.

c : Modificar la orientación aplicando un giro angular en sentido de las agujas del reloj (horario): $\theta \leftarrow \theta - \alpha$.

g : Modificar la orientación aplicando un giro angular en sentido inverso a las agujas del reloj (antihorario): $\theta \leftarrow \theta + \alpha$.

t : Operador de bifurcación de pila (Push/Pop). El primer encuentro almacena el estado (x, y, θ) en memoria; el segundo encuentro restaura dicho estado, permitiendo ramificaciones sin alterar la trayectoria principal.

Definición Formal de la Gramática G

La Gramática Libre de Contexto propuesta queda definida formalmente por la tétada $G = (V, \Sigma, R, S)$, donde:

1. Conjunto de Variables o No Terminales (V): $V = \{S, A, B, C, D\}$

2. Alfabeto de Terminales (Σ): $\Sigma = \{a, c, g, t\}$

3. Símbolo Inicial (S): Variable de arranque que inicializa el proceso de derivación.

4. Conjunto de Reglas de Producción (R):

Variable	Regla de Producción	Propósito Semántico
S	$S \rightarrow A \mid B \mid C \mid D$	Selector del tipo de patrón gráfico.
A	$A \rightarrow acacacac$	Generador de topologías ortogonales cerradas.

B	$B \rightarrow t a B a t g B$	Estructura fractal autorecursiva (Ramificación).
C	$C \rightarrow A g A c A$	Proyección volumétrica pseudo-3D.
D	$D \rightarrow a c a g D a$	Patrón alternante lineal (Secuencia oscilatoria).

2.2.2.- Ejemplos de Derivación de Producciones Formales

Ejemplo A: Derivación para un Cuadrado Perfecto

Configuración geométrica: Ángulo de rotación $\alpha = 90^\circ$. Distancia $d = 5$ unidades.

Derivación Formal:

$S \Rightarrow A$

$A \Rightarrow acacacac$

Cadena terminal obtenida: acacacac

Análisis de Ejecución Secuencial:

1. a: Traza la arista inferior horizontal.
2. c: Gira 90° en sentido horario.
3. a: Traza la segunda arista perpendicular.
4. c: Gira 90° en sentido horario.
5. a: Traza la tercera arista.
6. c: Gira 90° en sentido horario.
7. a: Traza la cuarta arista, cerrando el polígono en el origen.
8. c: Gira 90° finales, restaurando la dirección inicial.

Ejemplo B: Árbol Estructurado con Varias Ramas y Hojas

Configuración geométrica: Ángulo de rotación $\alpha = 45^\circ$. El símbolo t opera como delimitador de entorno.

Derivación Formal (2 niveles de recursividad):

$S \Rightarrow B$

$B \Rightarrow t a B a t g B$ (Aplicando $B \rightarrow t a B a t g B$)

$B \Rightarrow t a (a) a t g B$ (Sustituyendo el primer no terminal B por el caso base 'a')

$B \Rightarrow t a a a t g (t a B a t g B)$ (Expandiendo el segundo B recursivamente)

$B \Rightarrow t a a a t g t a (a) a t g (a)$ (Cierre de todas las variables con el caso base 'a')

Cadena terminal obtenida: taaatgtaaataga

Análisis de Ejecución Secuencial:

- El primer bloque t...t resguarda el tronco principal, proyectando un segmento de crecimiento rectilíneo (aaa).
- Al procesar el segundo t, la tortuga experimenta un retorno elástico hacia el nodo de bifurcación inicial.
- El comando g desvía el eje angular en $+45^\circ$ hacia la izquierda, abriendo la sección superior de la copa donde la subcadena interna taaatga genera brotes secundarios o 'hojas'.

Ejemplo C: Cubo Isométrico (Modelado Tridimensional)

Configuración geométrica: Ángulo de inclinación axonométrica $\alpha = 60^\circ$.

Derivación Formal:

$S \Rightarrow C$

$C \Rightarrow A g A c A$ (Estructura base de interconexión planar)

$C \Rightarrow (acacacac) g A c A$ (Sustitución de la primera variable planar)

$C \Rightarrow acacacac g (acacacac) c A$ (Sustitución de la segunda variable planar)

$C \Rightarrow acacacacgacacacacc(acacacac)$ (Sustitución final de la tercera variable planar)

Cadena terminal obtenida: acacacacgacacacaccacacacac

Análisis de Ejecución Secuencial: El sistema grafica de manera secuencial tres paralelogramos adyacentes. La primera subcadena dibuja la cara frontal; la transición g modifica el plano de incidencia angular para permitir que la segunda subcadena trace la cara superior en perspectiva; finalmente, el comando c reorienta el flujo sintáctico para plasmar la cara lateral derecha, conformando el volumen del cubo por solapamiento de aristas compartidas.

Ejemplo D: Línea en Zig-Zag (Patrón de Control de Texturas)

Configuración geométrica: Ángulo de rotación $\alpha = 90^\circ$.

Derivación Formal:

$S \Rightarrow D$

$D \Rightarrow a c a g D$ (Primera iteración lineal)

$D \Rightarrow a c a g (a c a g D)$ (Segunda iteración lineal)

$D \Rightarrow a c a g a c a g (a)$ (Sustitución por el caso base de parada)

Cadena terminal obtenida: acagacaga

Análisis de Enrutamiento: Produce un patrón ortogonal simétrico en forma de escalera. El operador c genera un quiebre descendente y el operador g restaura la horizontalidad constructiva, un método utilizado para el escaneo de superficies o tramados de relleno de figuras.

Ejemplo E: Triángulo Equilátero Perfecto

Configuración geométrica: Ángulo complementario exterior de giro $\alpha = 120^\circ$.

Derivación Formal: Regla secundaria derivada $A3 \rightarrow acacac$.

$S \Rightarrow A3$

$A3 \Rightarrow acacac$

Cadena terminal obtenida: acacac

Análisis Vectorial:

1. a: Dibuja la base del triángulo horizontal hacia la derecha.
2. c: Ejecuta una rotación física de 120° , dejando el cursor orientado en ángulo agudo interno de 60° .
3. a: Grafica la arista diagonal ascendente izquierda.
4. c: Rota nuevamente 120° en sentido horario.
5. a: Traza la diagonal descendente, intersectando con precisión absoluta el punto inicial.
6. c: Giro de convergencia para reestablecer la orientación original.

Conclusiones del Modelado Genómico-Geométrico

La implementación de Gramáticas Libres de Contexto asociadas a reglas semánticas espaciales demuestra que el alfabeto biológico del ADN es perfectamente capaz de codificar morfologías complejas y estructuradas. Este marco conceptual provee las bases esenciales para el desarrollo de algoritmos de optimización tipológica, síntesis gráfica automatizada y compresión de estructuras fractales utilizando cadenas minimalistas de caracteres de procesamiento secuencial.

2.3 Higiene y Optimización de Gramáticas

En el diseño de compiladores, la higiene y optimización de gramáticas consiste en transformar una Gramática Libre de Contexto (GLC) para eliminar fallas estructurales que impidan su análisis eficiente. Los algoritmos de análisis sintáctico deterministas, ya sean descendientes o ascendientes, requieren que la gramática cumpla con ciertas propiedades estrictas para poder tomar decisiones unívocas al leer los tokens de entrada.

Patologías de las Gramáticas

Las patologías son defectos de diseño en las reglas de producción. No significan que la gramática esté matemáticamente incompleta, sino que no es práctica para un compilador real, ya que causan bucles infinitos, **conflictos de decisión** (*lookahead*) o interpretaciones semánticas erróneas.

A continuación, se desarrollan en profundidad los tres casos prácticos solicitados:

a) Caso 1: Una Gramática Ambigua

Una gramática es ambigua si existe al menos una cadena en su lenguaje que puede ser generada a través de dos o más árboles de derivación sintáctica distintos. Para un compilador esto es catastrófico: el árbol sintáctico define la semántica y el orden de ejecución del código; si hay dos árboles, el programa tiene dos significados posibles.

- **Gramática Propuesta** :Expresiones aritméticas con suma y multiplicación.

$E \rightarrow E + E \mid E \times E \mid \text{id}$

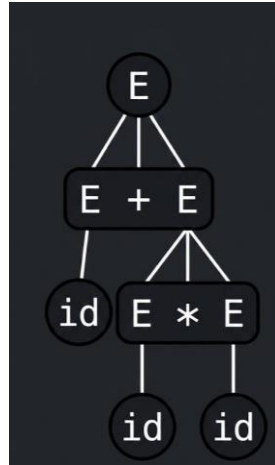
- **Cadena a evaluar:** $\text{id} + \text{id} * \text{id}$

Demostración de Ambigüedad (Árboles de Derivación)

Para demostrar la ambigüedad, construimos dos árboles sintácticos estructuralmente diferentes para la misma secuencia de tokens:

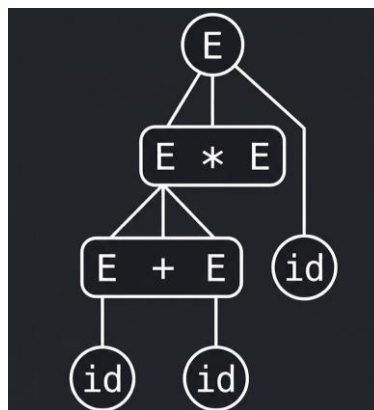
- **Árbol de Derivación A (Prioridad correcta de la multiplicación):**

Este árbol agrupa primero la multiplicación. En la evaluación bottom-up, $\text{id} * \text{id}$ se resuelve antes de sumarse con el primer id.



- **Árbol de Derivación B** (Prioridad incorrecta / Evaluación de izquierda a derecha):

Este árbol agrupa primero la suma. Al compilar, obligaría a ejecutar la suma antes que la multiplicación, rompiendo las reglas de la aritmética convencional.



Solución teórica: Para sanar esta patología, se debe reescribir la gramática estratificando los niveles de precedencia mediante nuevos no terminales (introduciendo conceptos como Término y Factor), forzando un único árbol posible.

b) Caso 2: Recursividad por la Izquierda La recursividad por la izquierda ocurre cuando un símbolo no terminal $A \rightarrow A\alpha$ se deriva en una producción que comienza por sí mismo de forma inmediata: Los analizadores sintácticos descendentes (como los de Descenso Recursivo) o entran en un bucle infinito de asignación de memoria (Stack Overflow). Al intentar expandir A, el parser vuelve a invocar la función de A de manera indefinida sin llegar a consumir ningún token real de la entrada que le permita avanzar. Gramática con Patología:

- Gramática con Patología:

$E \rightarrow E+T \mid T$

Aquí, $A=E$, la parte recursiva es $\alpha=+T$, y la base no recursiva es $\beta=T$.

Algoritmo Paso a Paso para Eliminar la Recursividad Izquierda

El algoritmo matemático consiste en transformar la recursión izquierda en una recursión derecha equivalente, introduciendo un nuevo símbolo auxiliar (A') que maneje de forma iterativa o final los sufijos remanentes. Dada la estructura general:

$A \rightarrow A\alpha \mid \beta$ Se transforma en: $A \rightarrow \beta A' \mid \alpha A' \mid \epsilon$ (Donde ϵ representa la cadena vacía).

Aplicación práctica al caso:

Paso 1: Identificar los componentes de la regla.

Analizando $E \rightarrow E+T \mid T$, determinamos que: El símbolo a corregir es $A=E$. El prefijo recursivo es $\alpha=+T$. La alternativa de escape (no recursiva) es $\beta=T$.

Paso 2: Reescribir el símbolo original (E).

Obligamos a que la expresión comience estrictamente por la estructura base β , seguida de un nuevo estado de control E' . $E \rightarrow TE'$

Paso 3: Definir el comportamiento del nuevo símbolo (E').

Este símbolo procesará el resto de la cadena en forma recursiva por la derecha ($\alpha E'$), o bien terminará limpiamente si ya no encuentra más operandos (ϵ). $E' \rightarrow +TE' \mid \epsilon$
Gramática Resultante Optimizada: $E \rightarrow TE' \mid E' \rightarrow +TE' \mid \epsilon$

Caso 3: Factorización por la Izquierda

La necesidad de factorización surge cuando un no terminal tiene dos o más reglas de producción que comienzan con el mismo prefijo de tokens. Un analizador predictivo LL(1) toma decisiones evaluando un solo token hacia adelante (lookahead). Si se encuentra con el prefijo común, el parser se queda "ciego": no puede saber matemáticamente cuál de las dos rutas tomar hasta no avanzar más allá, rompiendo el determinismo.

- Gramática con Patología (El problema del "Dangling Else"):

$S \rightarrow \text{if } e \text{ then } S \text{ else } S \mid \text{if } e \text{ then } S \mid \text{other}$

Ambas estructuras condicionales inician exactamente con el prefijo común $\alpha = \text{if } e \text{ then } S$.

Solución: Gramática Resultante Optimizada.

Para solucionarlo, aplicamos factorización por la izquierda. Extraemos el prefijo común α como un bloque único y postergamos la decisión sintáctica derivando el residuo en un nuevo no terminal (S').

- Identificamos el prefijo común: $\alpha = \text{if } e \text{ then } S$.
- Extraemos el prefijo en la regla de S , apuntando hacia la extensión indecisa S'

$S \rightarrow \text{if } e \text{ then } SS' \mid \text{other}$

- Definimos las ramificaciones reales dentro de S' (la opción que contiene el else y la opción que simplemente finaliza la instrucción sin hacer nada):

$S' \rightarrow \text{else } S \mid \epsilon$

- Gramática Resultante Optimizada:

$S \rightarrow \text{if } e \text{ then } SS' \mid \text{other } S' \rightarrow \text{else } S \mid \epsilon$

Ahora, cuando el parser lee el token if, sabe exactamente que debe aplicar la primera regla. Una vez que procesa todo el bloque del then, llegará al estado S' y mirará el siguiente token: si es un else, tomará la primera alternativa de S' ; si es cualquier otra cosa, asumirá la transición vacía (ϵ) de forma completamente determinista.

2.4) De la Expresión al Autómata (Caso Práctico: Ajedrez).

2.4.1.- Notación PGN (Portable Game Notation): Plantee y ejecute autómatas finitos para el reconocimiento de un subconjunto del lenguaje PGN (Ajedrez).

Planteamos un Autómata Finito Determinista (AFD) para reconocer movimientos estándar: Peón (e4), Pieza (Nf3), Captura (Bxe5) y Enroque corto (O-O).

ALFABETO: (Σ): {K,Q,R,B,N,a...h,1...8,x,O,-}

Este autómata reconoce un subconjunto del estándar PGN:

- Movimientos de peón (ej: e4)
- Movimientos de pieza (ej: Nf3)
- Capturas (ej: Bxe5)
- Enroque corto (O-O)

1. DEFINICIÓN FORMAL DEL AUTÓMATA (AFD)

$M = \{Q, \Sigma, \delta, q_0, F\}$

Donde:

Q (Estados): $\{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7\}$

Σ (Alfabeto): $\{K, Q, R, B, N, a-h, 1-8, x, O, -\}$

q_0 : Estado Inicial

F (Estados de Aceptación): $\{q_4, q_7\}$

2. TABLA DE TRANSICIONES

Estado Actual | Entrada | Próximo Estado | Descripción

q_0 | $[K, Q, R, B, N]$ | q_1 | Pieza identificada

q_0 | $[a-h]$ | q_2 | Columna (Peón)

q_0 | O | q_5 | Inicio Enroque

q_1 | $[a-h]$ | q_2 | Columna (Pieza)

q_1 | x | q_3 | Captura por pieza

q_2 | $[1-8]$ | q_4 (FINAL) | Fila (Mov. completado)

q_3 | $[a-h]$ | q_2 | Destino tras captura

q_5 | $-$ | q_6 | Separador enroque

q_6 | O | q_7 (FINAL) | Enroque completado

3. EJECUCIÓN DE TRAZAS

Ejemplo A: Movimiento "Nf3" (Pieza a casilla)

1. $\delta(q_0, N) \rightarrow q_1$

2. $\delta(q_1, f) \rightarrow q_2$

3. $\delta(q_2, 3) \rightarrow q_4$ [ACEPTADA]

Ejemplo B: Movimiento "e4" (Peón a casilla)

1. $\delta(q_0, e) \rightarrow q_2$

2. $\delta(q_2, 4) \rightarrow q_4$ [ACEPTADA]

Ejemplo C: Movimiento "Bxe5" (Captura)

1. $\delta(q_0, B) \rightarrow q_1$

2. $\delta(q_1, x) \rightarrow q_3$

3. $\delta(q_3, e) \rightarrow q_2$

4. $\delta(q_2, 5) \rightarrow q_4$ [ACEPTADA]

Ejemplo D: Movimiento "O-O" (Enroque corto)

1. $\delta(q_0, O) \rightarrow q_5$

2. $\delta(q_5, -) \rightarrow q_6$

3. $\delta(q_6, O) \rightarrow q_7$ [ACEPTADA]

4. REPRESENTACIÓN LÓGICA (PSEUDOCÓDIGO)

Si entrada es Pieza $\rightarrow$ Ir a q_1

Si entrada es Letra(a-h) $\rightarrow$ Ir a q_2

Si q_1 recibe Letra $\rightarrow$ Ir a q_2

Si q_1 recibe 'x' $\rightarrow$ Ir a $q_3 \rightarrow$ Luego Letra $\rightarrow$ Ir a q_2

Si q_2 recibe Número(1-8) $\rightarrow$ ESTADO FINAL (Movimiento Válido)

Si q_0 recibe 'O' $\rightarrow$ '-' $\rightarrow$ 'O' $\rightarrow$ ESTADO FINAL (Enroque Válido)

2.4.2.- Defina un subconjunto simplificado del PGN para movimientos básicos.

Para no enredarnos en casos borde, vamos a definir un subconjunto extremadamente estricto. Al principio dudé sobre si meter las capturas (la famosa "x" en el PGN), pero la verdad es que si las incluimos empezamos a ramificar mucho el autómata prematuramente. Mejor mantengámoslo estrictamente en movimientos básicos de desplazamiento de piezas y peones.

- **Reglas del subconjunto simplificado:**
 - **Movimiento de Peón:** En PGN, a los peones no se les asigna una letra. Simplemente se anota la casilla de destino. Consiste en una letra de columna (de la a a la h) seguida de un número de fila (del 1 al 8). Ejemplo: e4, d5.
 - **Movimiento de Pieza:** Se indica con la letra mayúscula representativa en inglés: K (King/Rey), Q (Queen/Reina), R (Rook/Torre), B (Bishop/Alfil), N (Knight/Caballo). A esto le sigue inmediatamente la casilla de destino. Ejemplo: Nf3, Bc4.

2.4.3. Diseñe la Expresión Regular (Regex) y el Autómata Finito Determinístico (AFD) equivalente.

Ahora que tenemos el alcance bien delimitado, armar la expresión regular es un proceso mucho más directo. Si observamos el patrón del subconjunto, todo movimiento termina inevitablemente en una coordenada (columna + fila). La única variación real ocurre al principio: si es una pieza, lleva mayúscula; si es un peón, arranca directo con la minúscula.

- **Expresión Regular Propuesta:**

`(K|Q|R|B|N)?(a|b|c|d|e|f|g|h)(1|2|3|4|5|6|7|8)`

- **Demostración de funcionamiento:**
 - **Caso Peón (e4):** El evaluador de la regex lee el primer grupo `(K|Q|R|B|N)?`. Como tiene el operador `?`, asume que es opcional y lo

omite. Luego hace match (coincide) con la e en el segundo grupo y con el 4 en el tercer grupo. La cadena es válida.

- **Caso Caballo (Nf3):** Hace match con la N obligatoria del primer grupo, pasa al segundo grupo tomando la f, y finaliza en el tercer grupo con el 3.
- **Nota mental:** A nivel de programación moderna uno simplemente escribiría `[KQRBN]?[a-h][1-8]`, que es mucho más limpio. Pero en teoría pura de lenguajes formales, expresarlo con las uniones (`|`) y concatenaciones es la forma matemáticamente estricta de definir el alfabeto.

El Autómata Finito Determinístico (AFD)

Para que un programa (o un analizador léxico) valide este lenguaje en tiempo real, necesitamos traducir esa Regex a un AFD. Un autómata determinista no puede dudar; al leer cada carácter (token) de entrada, debe tener un único camino posible hacia el siguiente estado. No podemos usar transiciones vacías (épsilon) porque romperíamos el determinismo.

- **Algoritmo Paso a Paso para la Construcción del AFD:**

Paso 1: Identificar los estados necesarios.

Pensando en la secuencia lógica de lectura:

- Necesitamos un estado inicial de arranque (q_0).
- Un estado por si leemos una pieza mayúscula (q_1).
- Un estado intermedio cuando ya leímos la letra de la columna (q_2).
- Un estado final que se alcance al leer el número de la fila (q_3).

Paso 2: Transiciones de inicio (Resolviendo la opcionalidad).

Aquí está el truco. Estando en el estado inicial q_0 :

- Si el usuario ingresa una pieza (K, Q, R, B, N), saltamos al estado q1.
- ¿Pero qué pasa si es un peón? Como no hay mayúscula, desde el mismo q0 debemos permitir que, si entra una letra de columna (a, b, c, d, e, f, g, h), salte directamente al estado q2. Esto evita usar saltos vacíos y mantiene el determinismo puro.

Paso 3: Cerrar los caminos hacia la aceptación.

- Si estamos en q1 (ya leímos la pieza), lo único legal que el usuario puede teclear después es la columna. Así que, ante cualquier entrada del conjunto (a, b, c, d, e, f, g, h), hacemos una transición hacia q2.
- Una vez en q2 (ya tenemos la pieza y la columna, o solo la columna del peón), nos falta la fila para terminar. Ante cualquier entrada del conjunto (1, 2, 3, 4, 5, 6, 7, 8), transitamos hacia q3.
- El estado q3 es nuestro Estado de Aceptación (generalmente dibujado con un doble círculo).

• Resumen del AFD Resultante Optimizado:

- q0 --- K, Q, R, B, N ---> q1
- q0 --- a, b, c, d, e, f, g, h ---> q2
- q1 --- a, b, c, d, e, f, g, h ---> q2
- q2 --- 1, 2, 3, 4, 5, 6, 7, 8 ---> q3 (Final)

Cualquier otro carácter que no esté especificado en estas transiciones desde el estado en el que se encuentre la máquina (por ejemplo, teclear un 9, una Z, o meter dos letras de pieza seguidas) enviará al autómata a un estado sumidero (estado de error), rechazando la cadena de manera automática y determinista.

Conclusión

El estudio de la teoría de lenguajes formales, las estructuras gramaticales y los modelos de autómatas trasciende la mera abstracción matemática para consolidarse como la columna vertebral de la informática moderna. Estos conceptos no solo definen las reglas del juego para la comunicación entre el ser humano y la máquina, sino que establecen los límites fundamentales de lo que es computable y de cómo se puede procesar la información de manera eficiente y segura.

La Jerarquía de Chomsky y el modelado de gramáticas demuestran que la sintaxis de cualquier lenguaje ya sea de programación, de control gráfico o de notación lógica requiere un equilibrio crítico entre el poder expresivo y la viabilidad técnica. Una gramática bien definida es el único medio que garantiza que un sistema de software interprete las instrucciones con absoluta fidelidad. Por el contrario, el análisis de las patologías sintácticas evidencia que las fallas de diseño en estos cimientos lógicos comprometen directamente la estabilidad de las herramientas de desarrollo, provocando ambigüedades interpretativas o colapsos por recursión que inutilizan los algoritmos de análisis. La higiene y optimización gramatical no son, por tanto, simples refinamientos estéticos, sino requisitos de ingeniería indispensables para asegurar la predictibilidad y el rendimiento de los compiladores e intérpretes.

Finalmente, la correspondencia entre expresiones regulares y autómatas finitos determinísticos simplifica la complejidad estructural, reduciendo comportamientos y flujos de datos complejos a mecanismos lógicos e invariables en el tiempo. En última instancia, la verdadera importancia de dominar estos formalismos radica en su capacidad para actuar como un puente universal: permiten transformar reglas abstractas y lenguajes de alto nivel en secuencias de estados físicos ejecutables por el hardware de una computadora, constituyendo un pilar invisible pero vital sobre el cual se edifica todo el software contemporáneo.

Referencias

- Grune, D., & Jacobs, C. J. (2008). Parsing Techniques: A Practical Guide (2.^a ed.). Springer Science & Business Media.
- Linz, P. (2016). An Introduction to Formal Languages and Automata (6.^a ed.). Jones & Bartlett Learning.
- Chomsky, N. (1956). Three models for the description of language. IRE Transactions on Information Theory, 2(3), 113-124.
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). Introducción a la Teoría de Autómatas, Lenguajes y Computación (3.^a ed.). Pearson Educación.
- Prusinkiewicz, P., & Lindenmayer, A. (2012). The Algorithmic Beauty of Plants. New York, NY: Springer Science & Business Media.
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). Introducción a la Teoría de Autómatas, Lenguajes y Computación (3a ed.). Madrid, España: Pearson Educación.